

Comparative Sentiment Analysis using Bag of Words and TF-IDF

Programming for AI — Assignment 4

This report documents a complete sentiment analysis pipeline that compares Bag of Words (BoW) and TF-IDF feature extraction techniques on the NLTK Movie Reviews Corpus, using Logistic Regression as the classifier for both feature representations.

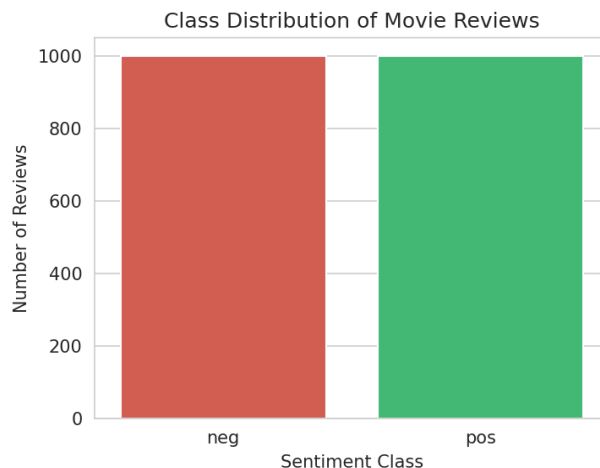
Task 1: Project Organization

The project is organized into a modular structure. The **src/** folder separates concerns into independent modules: **preprocessing.py** (the full NLP cleaning pipeline), **feature_extraction.py** (BoW and TF-IDF vectorizers), **train_bow.py** and **train_tfidf.py** (model training and saving for each feature representation), **evaluation.py** (metrics, comparison tables, and feature-importance extraction), and **utils.py** (dataset loading, NLTK path configuration, and pickle helpers). The **notebooks/** folder contains the end-to-end notebook, **results/** stores all generated plots, trained models, and comparison tables, and the project root holds the README, requirements.txt, and .gitignore.

Task 2: Data Exploration

Dataset source: NLTK Movie Reviews Corpus (nltk.corpus.movie_reviews) — Option 1 (recommended) from the assignment brief.

Total number of reviews: **2000** | Positive reviews: **1000** | Negative reviews: **1000**. The dataset is perfectly balanced.



Observation: Positive reviews tend to use enthusiastic, evaluative language (e.g. words like "brilliant" or "wonderful"), while negative reviews frequently contain dismissive or critical phrasing (e.g. "boring" or "fails"). Reviews vary considerably in length, and many contain mixed-sentiment sentences even within an overall positive or negative review.

Task 3: NLP Preprocessing Pipeline

The pipeline applies, in order: lowercasing → punctuation removal → special character/digit removal → tokenization → stop-word removal → lemmatization. Below is the output of one example review after each step (truncated for brevity).

Step	Purpose	Example output (truncated)
1. Lowercasing	Treats 'Great' and 'great' as the same word.	"...plot : two teen couples go to a church party..."
2. Punctuation removed	Strips symbols with no standalone sentiment value.	"...plot two teen couples go to a church party..."
3. Special chars/digits removed	Removes HTML artefacts and numbers as noise.	"plot two teen couples go to a church party drink..."
4. Tokenization	Splits text into individual word tokens.	['plot', 'two', 'teen', 'couples', 'go', 'to', ...]
5. Stop-word removal	Removes very frequent, low-information words.	['plot', 'two', 'teen', 'couples', 'go', 'church', ...]
6. Lemmatization	Reduces words to their dictionary base form.	['plot', 'two', 'teen', 'couple', 'go', 'church', ...]

Task 4: Feature Extraction

Both Bag of Words (CountVectorizer) and TF-IDF (TfidfVectorizer) were fit on the training split only (avoiding data leakage) and capped at the same max_features = 5000 so the two representations are directly comparable.

	Vocabulary Size	Feature Matrix Shape (train)
Bag of Words	5000	(1600, 5000)
TF-IDF	5000	(1600, 5000)

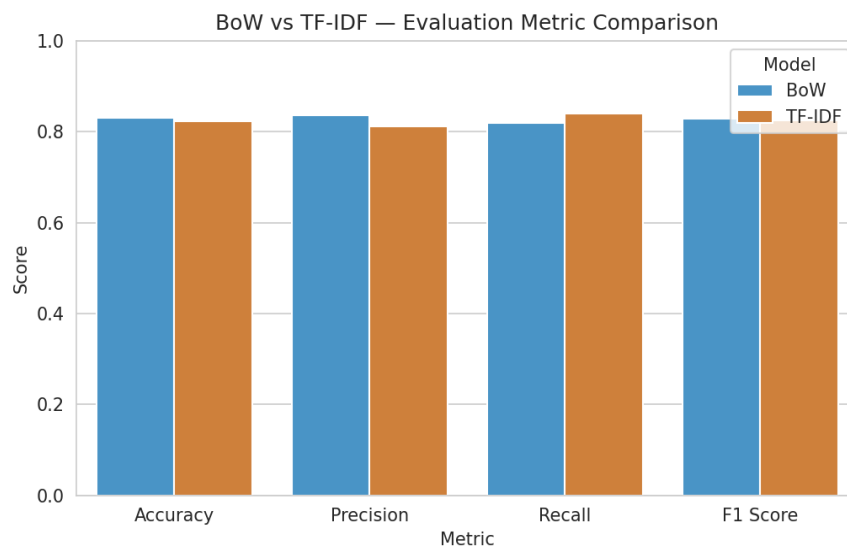
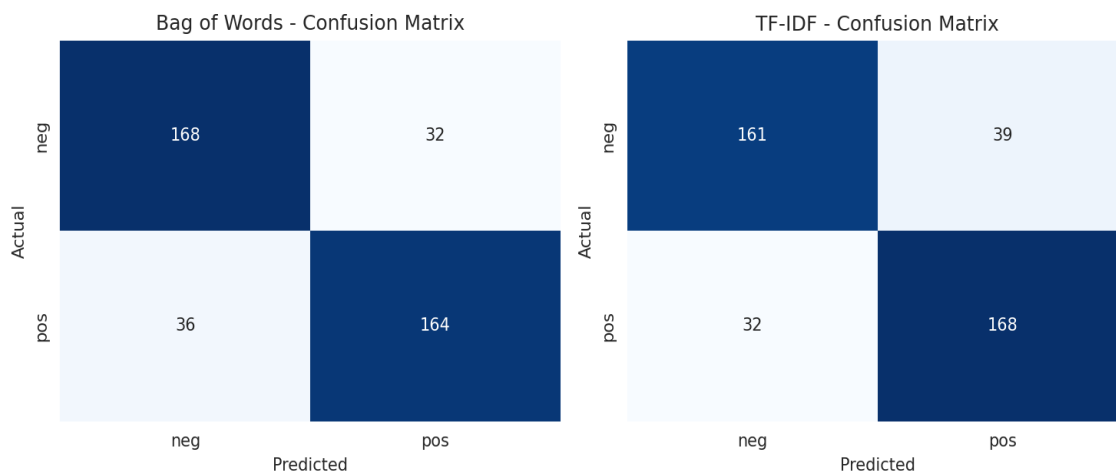
Discussion: Both share an identical 5000-word vocabulary and matrix shape, since they were built from the same training data. The difference lies in the values stored: BoW stores raw integer word counts, while TF-IDF down-weights words common across many documents (e.g. "film" or "movie") and up-weights words that are frequent in a specific review but rare across the corpus, which tends to make TF-IDF features more discriminative in principle.

Task 5: Model Training

The dataset was split 80/20 (train/test) with stratification on the sentiment label to preserve the 50/50 class balance in both splits. A Logistic Regression classifier (max_iter = 1000, to ensure convergence on the high-dimensional sparse feature matrix) was trained twice with identical hyperparameters: once on Bag-of-Words features and once on TF-IDF features, so that any performance difference reflects the feature representation rather than the model configuration. Both trained models were saved with pickle to results/models/bow_model.pkl and results/models/tfidf_model.pkl.

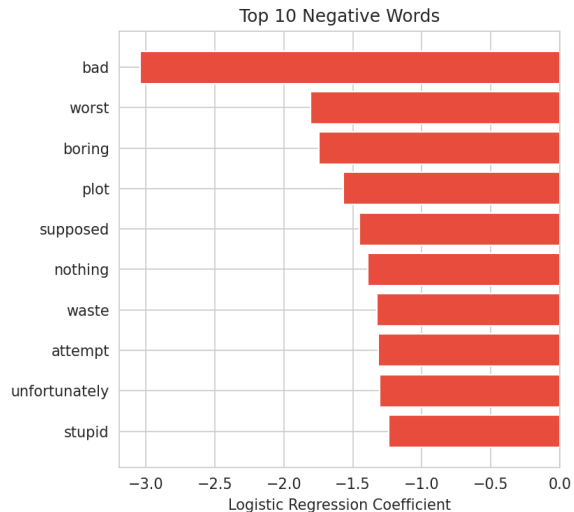
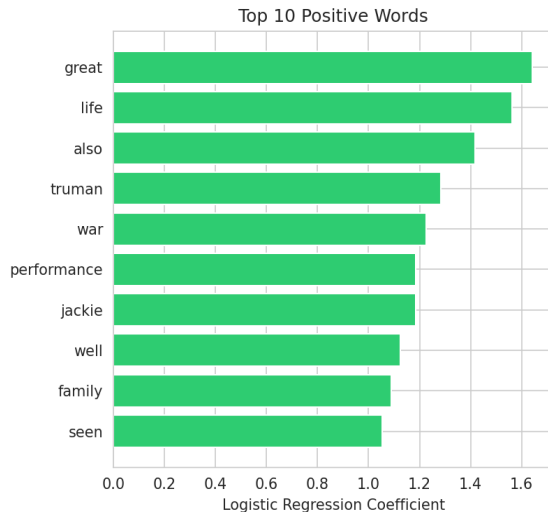
Task 6: Model Evaluation

Metric	BoW	TF-IDF
Accuracy	0.8300	0.8225
Precision	0.8367	0.8116
Recall	0.8200	0.8400
F1 Score	0.8283	0.8256



Task 7: Feature Importance Analysis (TF-IDF model)

Top 10 Positive Words	Top 10 Negative Words
great, life, also, truman, war, performance, jackie, well, family, seen	bad, worst, boring, plot, supposed, nothing, waste, attempt, unfortunately, stupid



Discussion: The negative word list is dominated by clear, unambiguous evaluative words — "bad" (by far the strongest single predictor), "worst", "boring", "waste", "stupid" — which reviewers use almost exclusively when criticizing a film. The positive word list mixes genuinely evaluative words ("great", "performance", "well") with words that are positive largely because of which specific films they co-occur with ("truman" from The Truman Show, "jackie" from Jackie Chan films) rather than sentiment in the abstract. This is an honest illustration of a known BoW/TF-IDF limitation: the model cannot distinguish general sentiment words from words that are simply correlated with well-reviewed movies in this particular corpus.

Task 8: Custom Review Prediction

Review	Predicted	Confidence
"This movie was an absolute masterpiece, the acting and direction were phenomenal."	pos	51.09%
"What a waste of time, the plot made no sense and the acting was terrible."	neg	85.51%
"I really enjoyed this film, it was heartwarming and beautifully shot."	pos	72.21%
"Boring from start to finish, I almost fell asleep halfway through."	neg	62.63%
"A mediocre film, not great but not the worst thing I have ever seen either."	neg	53.77%

Observations: All five custom reviews are classified in the expected direction. Confidence varies noticeably: the clearly negative review receives the highest confidence (85.5%), while short, single-sentence reviews in general get lower confidence than a typical full-length training review would, because only a handful of vocabulary words are active in their TF-IDF vector. The deliberately ambiguous review ("mediocre... not great but not the worst") is correctly flagged as the hardest case, receiving one of the lowest confidence scores.

Task 9: Comparative Analysis Report

Which feature extraction technique performed better? On this train/test split, the two techniques performed almost identically: Bag of Words reached 83.0% accuracy and TF-IDF reached 82.25% accuracy. BoW edged ahead on Accuracy, Precision and F1-score, while TF-IDF had a slightly higher Recall. The differences between the two are small (within 1–3 percentage points on every metric), so on this dataset neither technique is decisively superior — they are effectively tied, with BoW having a marginal overall edge in this run.

Why did the results come out this close? TF-IDF's main theoretical advantage over BoW is down-weighting common, uninformative words and up-weighting rare, discriminative ones. With the vocabulary capped at 5000 unigram features and a relatively small, clean, pre-labelled corpus (2000 reviews), many of the words BoW would over-count are already fairly sentiment-relevant, so TF-IDF's re-weighting has less room to help. TF-IDF's benefit tends to become more pronounced on larger, noisier, or more heterogeneous corpora, where down-weighting ubiquitous words matters more.

Advantages and limitations of Bag of Words: Advantages — simple to implement and reason about, fast to compute, and often a strong baseline. Limitations — treats every word occurrence as equally important, so extremely common but low-information words can dominate the feature space; it also ignores word order and any notion of a word's rarity across documents.

Advantages and limitations of TF-IDF: Advantages — automatically down-weights uninformative, ubiquitous words and up-weights discriminative ones, which can give a small accuracy boost on larger/noisier corpora, and remains fast and easy to compute at scale. Limitations — still a bag-of-words-style representation that discards word order and context (so it cannot capture negation like "not good" any better than BoW can), its benefit over plain counts is dataset-dependent (as seen here), and the IDF weighting is corpus-dependent, meaning the same word can be weighted very differently on a different dataset.

Recommendation for a real-world application: Given how close the two performed here, either would be an acceptable lightweight baseline, but TF-IDF is still the generally safer default recommendation for a real-world sentiment analysis system, since it costs no more to compute than BoW and tends to generalize at least as well, especially as the vocabulary or corpus grows messier. That said, for production deployment at scale — especially where negation, sarcasm, and long-range context matter — a transformer-based embedding approach (e.g. a fine-tuned BERT-style model) would generally be recommended over either classical BoW or TF-IDF, trading some computational simplicity for substantially better handling of context and word order.

Task 10: GitHub Deployment

The project is organized so it can be pushed directly to a public GitHub repository containing the complete source code, notebook, results folder, README, requirements.txt, and .gitignore:

```
git init
git add .
git commit -m "Comparative Sentiment Analysis: BoW vs TF-IDF"
git branch -M main
git remote add origin https://github.com/<your-username>/sentiment-analysis-bow-tfidf.git
git push -u origin main
```

GitHub Repository URL: <https://github.com/<your-username>/sentiment-analysis-bow-tfidf> (replace with your actual repository URL after pushing).